

Prácticas de Programación

PR2 - 20242

Fecha límite de entrega: **29 / 04 / 2025**

Formato y fecha de entrega

Hay que entregar la práctica antes del día **29 de abril 2025** a las 23:59.

Se deben entregar los archivos requeridos en la herramienta DSLab dentro del ejercicio PR2. Solo se valorará la última entrega dentro del plazo establecido.

Concretamente, en esta actividad se requieren los siguientes archivos:

- Un fichero **README.txt** con el siguiente formato (ver ejemplo):

Formato:

Correo electrónico UOC
Apellidos, Nombre
Sistema operativo utilizado

Ejemplo:

estudiantel@uoc.edu
Apellido1 Apellido2, Nombre
Windows 10

- Los ficheros de prueba sin modificaciones.
- Los ficheros *.c y *.h resultantes de los ejercicios realizados.
- Todos los ficheros deben estar dentro de las carpetas correctas (src, test, ...).
- Si se ha utilizado el entorno CodeLite: los ficheros .workspace y .project que definen el espacio de trabajo y los proyectos de Codelite.

Finalmente, también **se debe entregar un documento en formato PDF en el apartado de entregas de AC del aula de teoría que contenga un breve informe (máximo 2 páginas)** sobre el código desarrollado.

El incumplimiento del formato de entrega especificado anteriormente puede suponer un suspenso de la práctica.

Objetivos

- Saber interpretar y seguir el código de terceras personas.
- Saber compilar proyectos de código organizados en carpetas y librerías.
- Saber implementar un proyecto de código a partir de su especificación.

Criterios de corrección:

Cada ejercicio tiene asociada su puntuación sobre el total de la actividad. Se valorará tanto que las respuestas sean correctas como que también sean completas.

- No seguir el **formato de entrega**, tanto por lo que se refiere al **tipo y nombre de los ficheros** como al contenido solicitado, comportará una **penalización importante** o la cualificación con una **D de la actividad**.
- El código entregado **debe compilar para ser evaluado**. Si compila, se valorará:
 - Que **funcionen** tal como se describe en el enunciado.
 - Que obtenga el **resultado esperado** dadas unas condiciones y datos de entrada diseñadas (pruebas proporcionadas). No es necesario pasar todos los tests pero por lo menos debe mostrarse el resultado de estos por pantalla.
 - Que se respeten los **criterios de estilo** y que el código esté **debidamente comentado**. Se valorará especialmente el uso de comentarios en inglés.
 - Que las **estructuras** utilizadas sean las correctas.
 - Que se **separe correctamente la declaración e implementación** de las acciones y funciones, utilizando los ficheros correctos.
 - El **grado de optimización** en tiempo y recursos utilizados en la solución entregada.
 - Que se realice una **gestión de memoria** adecuada, liberando la memoria cuando sea necesario.

En esta práctica sólo se podrán modificar los archivos ***show.c***, ***show.h***, ***watchstack.c***, ***watchstack.h***, ***subscription.c***, ***api.c*** y ***api.h***, que serán los archivos a subir a la herramienta DSLab.

Aviso

Aprovechamos para recordar que **está totalmente prohibido copiar en las PECs y prácticas** de la asignatura. Se entiende que puede haber un trabajo o comunicación entre los estudiantes durante la realización de la actividad, pero la entrega de esta debe que ser individual y diferenciada del resto. Las entregas serán analizadas con **herramientas de detección de plagio**.

Así pues, las entregas que contengan alguna parte idéntica respecto a entregas de otros estudiantes serán consideradas copias y todos los implicados (sin que sea relevante el vínculo existente entre ellos) suspenderán la actividad entregada.

Guía citación: <https://biblioteca.uoc.edu/es/contenidos/Como-citar/index.html>

Monográfico sobre plagio:

<http://biblioteca.uoc.edu/es/biblioguias/biblioguia/Plagio-academico/>

Observaciones

En este documento se utilizan los siguientes símbolos para hacer referencia a los bloques de diseño y programación:



Indica que el código mostrado es en lenguaje **algorítmico**.



Indica que el código mostrado es en **lenguaje C**.



Muestra la **ejecución** de un programa en lenguaje C.

Análisis dinámico

En esta actividad se utiliza memoria dinámica, que requiere que el programador reserve, inicialice y libere la memoria. Para ayudar a detectar memoria que no se ha liberado correctamente, o errores en las operaciones con punteros relacionadas, hay herramientas que ejecutan un análisis dinámico del programa. Una herramienta de código abierto muy empleada es Valgrind (<https://valgrind.org/>). La utilización de esta herramienta queda fuera del ámbito del curso.

Para entender el significado de los **códigos de error**, podéis consultar el siguiente enlace, donde encontraréis ejemplos de código que os ayudarán a entender cuando se dan estos errores:

<https://bytes.usc.edu/cs104/wiki/valgrind/>

DSLAb

Siguiendo con la misma dinámica que en la práctica anterior, se usará la herramienta DSLab (<https://sd.uoc.edu/dslab/>). Esta herramienta también se utiliza en otras asignaturas y tiene como objetivo:

- Proporcionar un entorno común en el que evaluar los ejercicios de codificación.

Os aconsejamos realizar envíos periódicos a la herramienta de los diferentes ejercicios de código, ya que os permitirá detectar posibles errores antes de la entrega final. **Tened presente que es la herramienta utilizada como base para corregir vuestros códigos, y que no se corregirá ningún código en otro entorno o máquina.** Así pues, si vuestro código no funciona en la herramienta DSLab, se considerará que no funciona, aunque lo haga en vuestro ordenador.

En todo caso, **hay que tener presente que las entregas finales deben seguir haciéndose en el apartado correspondiente del aula, tal como indica el enunciado.** Esta herramienta es una ayuda adicional que ponemos a vuestra disposición, y en ningún caso es obligatorio su utilización.

La información básica que os será de utilidad al utilizar DSLab:

- DSLab considera la entrega correcta únicamente si esta pasa todos los tests.
- Se muestra un resumen rápido del número de tests pasados. Por norma general no será necesario pasarlos todos para aprobar la entrega.
- En los detalles se muestra:
 - El detalle de los tests pasados y de los que han fallado.
 - Es posible descargar un fichero con el texto que el programa muestra por pantalla (salida estándar). Se ha incluido el uso de **valgrind** en la salida estándar de manera que en este apartado podréis ver el informe sobre la **gestión de la memoria**. Es recomendable revisar esta parte para asegurarse de que se hace un uso correcto de los punteros y de la memoria.
- Existe también un log de ejecución que guarda la evolución de la ejecución del programa. Si debido a una codificación incorrecta el programa falla y no es capaz de mostrar el resultado de los tests, se debe revisar este log para determinar en qué punto se interrumpió la ejecución.

Nota: aunque DSLab es un sistema robusto y utilizado en varias asignaturas de la UOC, para esta asignatura en concreto está en fase de pruebas. Si encontráis algún problema indicadlo a los profesores para que podamos corregir cualquier incidencia.

Enunciado

El punto de partida de esta práctica es la solución oficial de la PR1. El código base del enunciado incluye todos los archivos necesarios de las actividades anteriores, con los que veréis que todos los tests de la PR1 pasan correctamente.

En esta práctica utilizaremos la estructura de datos planteada en la PR1 con algunas modificaciones y nuevas estructuras con sus funciones relacionadas para poder gestionar los datos de las películas, series y seguimiento de películas.

La estructura **tCatalog** ha sido renombrada a **tFilmCatalog**, y se ha añadido una nueva para almacenar series llamada **tShowCatalog**.



type

```
tFilmstackNode = record
    elem : tFilm;
    next : pointer to tFilmstackNode;
end record

tFilmstack = record
    top : pointer to tFilmstackNode;
    count : integer;
end record

tEpisode = record
    title : pointer to char;
    number : integer;
    duration : tTime;
    rating : float;
end record

tEpisodeNode = record
    episode : tEpisode;
    next : pointer to tEpisodeNode;
end record

tEpisodeQueue = record
    first : pointer to tEpisodeNode;
    last : pointer to tEpisodeNode;
    count : integer;
end record

tSeason = record
    number : integer;
    releaseDate : tDate;
```

```

        numEpisodes : integer;
        episodes : tEpisodeQueue;
    end record

    tSeasonNode = record
        season : tSeason;
        next : pointer to tSeasonNode;
    end record

    tSeasonList = record
        first : pointer to tSeasonNode;
        count : integer;
    end record

    tShow = record
        name : pointer to char;
        seasons : tSeasonList;
    end record

    tShowNode = record
        show : tShow;
        next : pointer to tShowNode;
        prev : pointer to tShowNode;
    end record

    tShowCatalog = record
        first : pointer to tShowNode;
        last : pointer to tShowNode;
        count : integer;
    end record

end type

```

Ejercicio 1: Gestión de series [50%]

En este ejercicio trabajaremos con la gestión de series de televisión utilizando estructuras de datos en C. Se os proporcionará un conjunto de archivos con definiciones y funciones básicas relacionadas con series, temporadas y episodios. A partir de ese código base, deberéis completar la implementación de varios métodos clave que permitirán inicializar, añadir y consultar información dentro del catálogo de series.

Este ejercicio está diseñado para que pongáis en práctica el uso de estructuras dinámicas, la correcta gestión de memoria y la organización modular del código en un proyecto más complejo. Prestad atención a los requisitos de cada apartado, así como a los posibles errores que debéis contemplar en los casos indicados.

En el código proporcionado encontraréis los archivos ***shows.c*** y ***shows.h*** con la definición de los tipos de datos anteriores y los principales métodos relacionados.

En el fichero ***shows.c*** implementad:

- a) **PR2_1a:** El método ***showList_init*** que dada una estructura de tipo ***tShowCatalog***, la inicialice correctamente.
- b) **PR2_1b:** El método ***show_init*** que dada una estructura de tipo ***tShowCatalog***, la inicialice correctamente.
- c) **PR2_1c:** El método ***episodeQueue_enqueue*** que dada una estructura de tipo ***tEpisodeQueue*** y uno de tipo ***tEpisode***, añade el episodio a la cola de episodios
- d) **PR2_1d:** El método ***season_addEpisode*** que dada una estructura de tipo ***tShowCatalog***, un nombre de serie, un número de temporada y un tipo ***tEpisode***, añade el episodio a la serie. En caso de ya existir el episodio, se devolverá deberá devolver un error de tipo ***E_EPISODE_DUPLICATED***. En caso de finalizar correctamente, la función debe devolver ***E_SUCCESS***.
- e) **PR2_1e:** El método ***seasonList_add*** que dada una estructura de tipo ***tSeasonList*** y una estructura de tipo ***tSeason***, añade la nueva temporada al principio de la lista.
- f) **PR2_1f:** El método ***showList_add*** que dada una estructura de tipo ***tSeasonList*** y una estructura de tipo ***tShow***, está podrá contener una única temporada con un único episodio, que en caso de no existir, se añadirán también.
- g) **PR2_1g:** El método ***show_seasonTotalDuration*** que dada una estructura de tipo ***tShowCatalog*** y una estructura de tipo ***tShow***, un nombre de serie, un número de temporada, nos devuelve la duración de tiempo de la temporada en un tipo ***tTime***.
- h) **PR2_1h:** El método ***show_seasonAverageRating*** que dada una estructura de tipo ***tShowCatalog***, un nombre de serie y un número de temporada, nos devuelve la media de valoración de los episodios de esa temporada. Si no tuviera episodios la temporada o no tuviera episodios, nos devolverá ***0.0f***

- i) **PR2_1i**: El método ***showList_free*** , que dado un objeto de tipo **tShowCatalog**, elimina los datos que contiene.
- j) **PR2_1j**: El método ***showsList_len***, que dado un objeto de tipo **tShowCatalog**, devuelve el número de series que contiene.

En el código proporcionado también encontraréis la definición e implementación de otras funciones que os pueden ser de utilidad:

- El método ***season_init***, que dado un objeto de tipo **tSeason**, lo inicializa con los datos recibidos como parámetros.
- El método ***episode_init***, que dado un objeto de tipo **tEpisode**, lo inicializa con los datos recibidos como parámetros.
- El método ***show_cpy***, que realiza una copia de una estructura **tShow** junto a todos los los punteros dentro del show.
- El método ***season_cpy***, que realiza una copia de una estructura **tSeason** junto a todos los los punteros dentro del show.
- El método ***episode_cpy***, que realiza una copia de una estructura **tEpisode** junto a todos los los punteros dentro del show.
- El método ***show_free*** que dado un objeto de tipo **tShow** elimina todos los datos que contiene.
- El método ***seasonList_free***, que dado un objeto de tipo **tSeasonList**, elimina los datos que contiene.
- El método ***showList_print***, que imprime por pantalla la información de un objeto de tipo **tShowCatalog**.
- El método ***show_print***, que imprime por pantalla la información de un objeto de tipo **tShow**.

Ejercicio 2: Pila de seguimiento de películas [30%]

En muchas plataformas de streaming encuentras una funcionalidad llamada "lista de seguimiento" o "watchlist", donde puedes almacenar las películas que te interesan para verlas más adelante. Esta lista facilita que no se te olvide ningún título deseado y te permite acceder rápidamente a tus próximas opciones de visionado.

El siguiente ejercicio se centra en implementar una versión simplificada de este concepto, utilizando una pila (estructura LIFO) para gestionar la lista de películas. A través de este ejercicio, aprenderás a inicializar, modificar y liberar una pila de seguimiento asociada a una suscripción, replicando una de las funciones esenciales que ofrecen las plataformas para una experiencia de usuario más organizada y personalizada.

En el código proporcionado encontraréis los ficheros **watchstack.h** y **watchstack.c** con la definición de los tipos de datos y los principales métodos relacionados con la gestión de la pila de seguimiento de películas para ver asociada a una suscripción.

En el fichero **subscription.c** modifica los siguientes métodos:

- a) **PR2_2a: subscriptions_add**: Completa la función para que inicialice la pila de seguimiento de películas (**tFilmstack**) asociada a una suscripción.
- b) **PR2_2b: subscription_free**: Completa la función para que libere la memoria ocupada por la pila de visionado (**tFilmstack**) asociada a una suscripción.

En el fichero **watchstack.c** implementa los siguientes métodos:

- c) **PR2_2c: filmstack_init**: Inicializa una estructura **tFilmstack** como una pila vacía. Devuelve **E_SUCCESS** si se inicializa correctamente.
- d) **PR2_2d: filmstack_isEmpty**: Indica si la pila que se le pasa está vacía (**true**) o, por el contrario, contiene elementos (**false**).
- e) **PR2_2e: filmstack_push**: Añade una película (**tFilm**) a la pila. Devuelve **E_SUCCESS** si se añade correctamente, si se intenta añadir uno repetido devolverá **E_FILM_ALREADY_EXISTS** y en caso de que no se pueda asignar memoria para el nuevo elemento devolverá **E_MEMORY_ERROR**.
- f) **PR2_2f: filmstack_top**: Devuelve un puntero al elemento (**tFilm**) que está en la cima de la pila. Este elemento no se eliminará de la pila. En caso de que la pila esté vacía, devuelve **NULL**.
- g) **PR2_2g: filmstack_pop**: Extrae el elemento que está en la cima de la pila y lo devuelve por referencia. Devuelve **E_SUCCESS** si la extracción se realiza correctamente, o **E_STRUCTURE_EMPTY** si la pila está vacía.
- h) **PR2_2h: filmstack_free**: Elimina todos los elementos de la pila y libera la memoria ocupada por ellos. Devuelve **E_SUCCESS** tras liberar correctamente la pila.

Ejercicio 3: Integración en la API [20%]

Finalmente queremos incorporar la gestión de las series y la pila de seguimiento de películas en la estructura del API, definida e implementada en los ficheros **api.h** y **api.c**. Se pide:

- a) **PR2_3a:** Completa la definición del tipo de datos **tApiData** del archivo **api.h** para que guarde la lista de series **tShowCatalog** definida en el ejercicio anterior.
- b) **PR2_3b:** Modifica la función **api_initData** del archivo **api.c** para que inicialice la lista de series.
- c) **PR2_3c:** Implementa la función **api_addToWatchlist** para que cuando se añada un nueva película a la lista de seguimiento para la id de una subscripción:
 - i) Añade una película a la suscripción.
 - ii) Si la película ya existe, no la añadirá.
- d) **PR2_3d:** Implementa la función **api_findShow** en el archivo **api.c**, que busca una serie por su nombre dentro del catálogo de series en la **tApiData** dada. Si no encuentra la serie devolverá NULL.
- e) **PR2_3e:** Modifica el método **api_addDataEntry** del archivo **api.c** para que cuando se añada un nuevo show:
 - i) Añade un show, temporada y episodio al catálogo de series.
 - ii) Si el show, la temporada o el episodio ya existen, no los añadirá.
- f) **PR2_3f:** Implementa el método **api_addShow** del archivo **api.c** que, dada una estructura de tipo **tApiData** y el nombre de una serie, almacene la información de la serie con dicho nombre en una estructura de tipo **tCSVEntry**.

El formato de la serie que debe devolver es el siguiente:

```
"showName;seasonNumber;seasonDate;episodeNumber;episodeTitle;
duration;rating"
```

Por ejemplo:

```
"SHOW;Breaking Bad;1;20/01/2008;1;Pilot;00:58;4.9"
"SHOW;Breaking Bad;1;27/01/2008;2;Cat's in the Bag...;00:48;4.8"
"SHOW;Stranger Things;1;15/07/2016;1;The Vanishing of Will
Byers;00:47;4.7"
"SHOW;Stranger Things;1;15/07/2016;2;The Weirdo on Maple
Street;00:55;4.6"
```

Para cada dato, será necesario comprobar que el formato es correcto (asumimos que es correcto si el número de campos es el esperado, error asociado **E_INVALID_ENTRY_FORMAT**) y que el tipo es "SHOW" (puedes acceder al tipo mediante el método `csv_getType`, error asociado **E_INVALID_ENTRY_TYPE**).

En caso de finalizar correctamente, la función debe devolver **E_SUCCESS**.

Nota: En los archivos **show.h** y **show.c** encontrarás una nueva definición de **show_parse** que devuelve la información de una serie en una estructura de tipo **tShow**. También encontrarás los métodos para gestionar las estructuras de datos de la serie **tShow**. Asimismo, recuerda que en el primer ejercicio implementaste el método **show_free** para eliminar la memoria dinámica reservada temporalmente para guardar los datos de la serie.

- g) **PR1_3g:** Implementa el método **api_showsCount** que dada una estructura de datos de tipo **tApiData**, retornen la cantidad de series.
- h) **PR1_3h:** Modifica el método **api_freeData** del archivo **api.c**, para que elimine todos los datos referentes a las series.